

Semantic Consistency as a Systems Invariant: Replay, Abstraction, and Event-Sourced Computation

Flyxion

December 14, 2025

Abstract

Operating systems, programming languages, and distributed systems traditionally ground correctness in state-based invariants, observational equivalence, or behavioral specifications. This paper develops an alternative foundation in which semantic consistency is defined over replayable event histories rather than instantaneous system states. Abstraction is treated as the stabilization of equivalence classes over construction histories, enforced by deterministic replay. We argue that many familiar system pathologies—state divergence, irreproducibility, brittle abstractions—arise from conflating correctness with state agreement rather than semantic invariance. By treating replay as an axiomatic constraint and abstraction as a semantic operation, we derive a systems model that cleanly separates authority from observation, causality from view, and construction from representation.

1 Introduction

Systems theory has long struggled with the tension between expressivity and correctness. As systems grow distributed, concurrent, and interactive, maintaining global invariants becomes increasingly difficult. Traditional approaches attempt to control this complexity through synchronization, consistency models, or increasingly sophisticated type systems. Yet many failures persist, not because systems compute incorrect values, but because their abstractions drift, fracture, or collapse under evolution.

This paper argues that the root cause lies in a category error: treating semantic correctness as a property of instantaneous system state rather than of construction history. State-based reasoning obscures causality, erases alternatives, and prevents principled abstraction. In contrast, event-sourced systems with deterministic replay make construction explicit and allow abstraction to be defined as an operation over histories.

The contribution here is not a new consistency model or synchronization protocol, but a reframing of what consistency means. Semantic consistency is defined as invariance under replay, not agreement on state. This shift enables abstractions that remain stable under refactoring, extension, and recomposition—properties increasingly essential in long-lived systems.

2 State, Events, and Authority

State-based systems treat the current configuration of memory as authoritative. Events are ephemeral; once applied, they vanish into state. This architecture obscures causality and makes it impossible to reconstruct why a system is in its present configuration without auxiliary logging.

Event-sourced systems invert this relationship. Events are authoritative; state is derived. However, most practical event-sourced architectures treat replay as a convenience for recovery or auditing, not as a semantic constraint. As a result, abstraction remains informal and brittle.

Axiom 2.1. *The authoritative substrate of a semantic system is an append-only, totally ordered event history.*

This axiom enforces a strict separation between causation and observation. State becomes a cache, not a source of truth.

Proposition 2.1. *Any system whose authoritative semantics are state-based cannot, in general, support revisable abstraction without external bookkeeping.*

The proof follows from the fact that abstraction requires comparison across alternative histories, which state erasure precludes.

3 Replay as a Semantic Constraint

Replayability is often framed as a debugging or recovery feature. Here it is treated as a semantic invariant: every abstract equivalence must be justifiable by replaying construction histories.

Deterministic replay ensures that the same event prefix always yields the same derived state. More importantly, it ensures that semantic equivalence can be tested by perturbing, reordering (where permitted), or partially replaying histories.

Definition 3.1. *A system is replay-consistent if all semantic claims made by the system are invariant under deterministic replay of its authoritative event history.*

Replay-consistency is stronger than observational equivalence and orthogonal to linearizability or serializability. Two systems may be observationally indistinguishable while differing semantically if their construction histories diverge.

4 Abstraction as Representative Collapse

Abstraction in systems is typically implemented implicitly: APIs hide details, types erase structure, modules encapsulate behavior. These mechanisms presume abstraction but do not define it.

In a replay-grounded system, abstraction is explicit. Multiple construction histories may be identified as semantically equivalent if they yield identical effects under all admissible replays. This identification induces a representative collapse: one semantic representative stands in for many histories.

Definition 4.1. *A semantic representative is an equivalence class of event histories invariant under replay.*

Representative collapse is neither compression nor optimization. It is a semantic act that reduces degrees of freedom in future reasoning while preserving invariance.

Proposition 4.1. *Representative collapse is irreversible without access to the original construction histories.*

This irreversibility explains why abstraction carries cost and why premature abstraction leads to brittleness.

5 Semantic Consistency vs. Correctness

Correctness traditionally refers to adherence to a specification. Semantic consistency refers to stability of meaning across system evolution.

A system may be correct at every step yet semantically inconsistent over time if abstractions drift. Conversely, a system may temporarily violate a specification while preserving semantic consistency.

Definition 5.1. *A system is semantically consistent if all abstractions remain invariant under admissible replay-preserving transformations.*

This definition decouples semantic reasoning from low-level correctness and aligns better with long-lived, evolving systems.

6 Concurrency, Distribution, and Total Order

Distributed systems often reject total order as impractical or unnecessary. However, rejecting total order at the semantic level conflates performance optimization with meaning.

This framework requires total order only at the level of authoritative events. Concurrency may exist in execution, but semantics are grounded in a single, replayable history.

Proposition 6.1. *Total semantic order is compatible with concurrent execution and does not imply synchronous operation.*

The total order functions as a semantic spine, not a scheduling constraint. This resolves many ambiguities in distributed abstraction.

7 Programming Languages Implications

Programming languages traditionally enforce abstraction through types, modules, and effects. These mechanisms can be reinterpreted as constraints on admissible event histories rather than as properties of state.

Types restrict which constructions are legal; effects constrain which events may occur; modules delimit replay scope. None of these mechanisms require representational semantics when grounded in event history.

This perspective suggests a reorientation of PL theory: from type soundness as state safety to type soundness as replay invariance.

8 Implementation Notes

A minimal system implementing these principles requires only: (1) an append-only event log, (2) a deterministic replay engine, (3) a mechanism for declaring and enforcing semantic equivalence.

Performance optimizations—snapshots, caching, parallel replay—are derived concerns. They must preserve replay-consistency but are otherwise unconstrained.

9 Limitations and Scope

This paper does not claim that replay-grounded semantics are sufficient for all systems. Real-time control, probabilistic systems, and hardware-level nondeterminism pose challenges.

However, these challenges arise at the boundary between semantics and physics, not from the framework itself. Where nondeterminism exists, it must be reified as events to preserve semantic reasoning.

10 Conclusion

By grounding semantics in replayable event histories, systems can support abstraction as a first-class operation rather than an emergent accident. Semantic consistency emerges as an invariant distinct from correctness, performance, or availability.

This reframing clarifies longstanding tensions in OS design, PL theory, and distributed systems, and provides a foundation for abstractions that survive evolution rather than collapsing under it.